

Администрирование Linux: Управление пакетами



Андрей
Тряпичников



Андрей Тряпичников

Системный администратор

Одноклассники

Предисловие

На этом занятии мы поговорим о:

- об устройстве пакета;
- о репозиториях;
- о менеджерах пакетов;
- об исходных файлах.

По итогу занятия вы узнаете, как управлять программным обеспечением, используя менеджеры пакетов.

План занятия

1. [Предисловие](#)
2. [Репозитории и пакеты](#)
3. [Пакетные менеджеры RPM, DNF и YUM](#)
4. [Пакетные менеджеры dpkg и APT](#)
5. [Компиляция программ и сборка пакетов](#)
6. [Прочие пакетные менеджеры](#)
7. [Итоги](#)
8. [Домашнее задание](#)



Репозитории и пакеты

Репозитории и менеджеры пакетов

Репозиторий — хранилище данных, в данном случае — пакетов ПО.

Менеджер пакетов — специальное программное обеспечение, которое управляет загрузкой, установкой, удалением пакетов, а также решением зависимостей.

Очень часто в системе пакетами управляет больше одного инструмента.



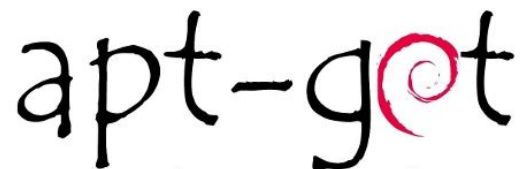
Для локальной работы

Пакетные менеджеры **rpm** (семейство Red Hat) и **dpkg** (семейство Debian) локальные: они не предназначены для скачивания пакетов из интернета. Они **управляют пакетами локально**: позволяют установить полученный вручную пакет, посмотреть список установленных пакетов и получить подробную информацию о них.



Для работы с репозиториями

Пакетные менеджеры **dnf** (и устаревший **yum**) (семейство Red Hat) и **apt** (семейство Debian) работают в первую очередь с удалёнными репозиториями в интернете и локальных сетях. Они скачивают пакеты и управляют зависимостями, отслеживают, в каких репозиториях какие пакеты есть и зачастую вызывают локальные пакетные менеджеры для установки и удаления пакетов.



Пакет

Пакет – это архив специального формата, который может содержать:

- бинарные и конфигурационные файлы и пути их размещения;
- данные о зависимостях пакета;
- список действий, которые необходимо выполнить в процессе установки.

Метapakет – пакет, объединяющий в качестве зависимостей набор пакетов, предназначенных для определённых целей. По сути такой пакет содержит только данные о зависимостях, и нужен для их установки.

Зависимости

Практически любая программа требует для своей работы дополнительные библиотеки и компоненты.

Такие **дополнительные материалы, необходимые для работы приложения**, устанавливаемого из пакетов, называют зависимостями.

Современные пакетные менеджеры решают эту проблему благодаря **списку зависимостей в пакете**.

Сторонние репозитории

Бывает так, что команда дистрибутива поддерживает устаревшую версию ПО в рамках эксплуатируемой нами версии дистрибутива.

В этом случае команда разработчиков этого ПО (например, языка программирования PHP или веб-сервера nginx) поддерживают свои репозитории с актуальными версиями своих программ.

Такие **репозитории** называют **сторонними**, так как они поддерживаются не в рамках версии дистрибутива.

Бывает, что в команде дистрибутива некому поддерживать пакет, и разработчик может размещать его в своём стороннем репозитории, чтобы он вообще был доступен пользователям дистрибутива.

Подключение сторонних репозиториев

В дистрибутивах семейства **Red Hat** конфигурационные файлы репозиториев хранятся в [/etc/yum.repos.d/](#). Нередко конфигурация репозитория поставляется в пакете. В этом случае его достаточно скачать и установить с помощью **rpm** или даже **dnf/yum**. Например, в репозиториях дистрибутивов на базе Red Hat можно установить конфигурацию репозиториев **epel** и **elrepo**.

В дистрибутивах семейства **Debian** репозитории конфигурируются в файле [/etc/apt/sources.list](#), а также в файлах в директории [/etc/apt/sources.list.d/](#).

В целях повышения безопасности многие репозитории поставляют gpg-ключ, которым подписаны пакеты. Иногда их нужно устанавливать вручную.

Подключение сторонних репозиториев

Разберем на примере CentOS и nginx:

1. Установим необходимое ПО:

```
sudo dnf install yum-utils
```

2. Добавим репозиторий в конфигурацию:

```
cat > /etc/yum.repos.d/nginx.repo <<EOF
[nginx-stable]
name=nginx stable repo
baseurl=http://nginx.org/packages/centos/\$releasever/\$basearch/
gpgcheck=1
enabled=1
gpgkey=https://nginx.org/keys/nginx_signing.key
module_hotfixes=true
EOF
```

А иногда достаточно такого (если конфигурация репозитория доступна в виде пакета):

```
sudo dnf install elrepo
```



Пакетные менеджеры RPM, DNF и YUM

Пакетный менеджер RPM

RPM (RPM Package Manager) — открытый менеджер пакетов для дистрибутивов Linux, предназначенный для управления пакетами в формате RPM.

С его помощью можно устанавливать пакеты, удалять их, отслеживать, какие пакеты установлены в системе, какие файлы к каким пакетам относятся и т.д., например:

- `rpm -i <file>` – установить пакета из указанного rpm файла;
- `rpm -e <pkg>` – удалить указанный пакет;
- `rpm -qa` – перечислить все установленные пакеты (есть поиск по строке);
- `rpm -qi <pkg>` – показать информацию об указанном пакете;
- `rpm -ql <pkg>` – перечислить файлы, принадлежащие указанному пакету;
- `rpm -qf <file>` – выяснить, к какому пакету относится указанный файл;

Пакетный менеджеры DNF и YUM

YUM (Yellowdog Updater, Modified) — открытый менеджер пакетов для дистрибутивов Linux, основанных на **пакетах формата RPM** (RedHat, CentOS, Fedora, Oracle Linux).

DNF (Dandified YUM) — новая версия YUM с полностью переписанной кодовой базой (с сохранением полной совместимости).

DNF осуществляет работу с **репозиториями** и **зависимостями**, скачивает пакеты и устанавливает их с помощью **rpm**.

Синтаксис построения команды

Пример:

dnf install -y vim

Где:

- **dnf** – пакетный менеджер;
- **install** – выполняемое действие;
- **-y** – ключ команды;
- **vim** – цель этой команды.

Обновление пакетов

- `yum update` – проводит обновление всех пакетов в системе, перед этим выполняя обновление списка пакетов в репозитории;
- `yum update <pkg>` – обновляет только выбранный пакет;
- `yum downgrade <pkg>` – откатывает пакет к предыдущей версии.

Информация о пакетах

- `yum search <pattern>` – полнотекстовый поиск в репозитории по паттерну;
- `yum list` – выводит список пакетов в зависимости от ключа:
 - `--installed` – установленных;
 - `--available` – всех доступных пакетов;
 - `--all` – всех доступных и установленных пакетов.

Установка и удаление пакета из репозитория

- `yum install <pkg>` – устанавливает в систему пакет, разрешая зависимости;
- `yum remove <pkg>` – удаляет файлы пакета из системы, но оставляет пользовательские файлы с настройками;
- `yum reinstall <pkg>` – переустанавливает пакет.

Полезные команды

- `yum autoremove` – удалить зависимости, которые больше не нужны;
- `yum clean packages` – удалит пакеты из кеша.



Пакетные менеджеры dpkg и APT

Пакетный менеджер `dpkg`

dpkg (debian package) — программа для управления пакетами в формате DEB в операционных системах **Debian** и основанных на них (**Ubuntu**, **Linux Mint** и т. п.).

Как и в случае с RPM, управляет именно файлами и базой пакетов, и предоставляет аналогичный rpm функционал, но работает с пакетами DEB:

- `dpkg -i <file>` – установить пакета из указанного deb файла;
- `dpkg -r <pkg>` – удалить указанный пакет;
- `dpkg -l` – перечислить все установленные пакеты (есть поиск по строке);
- `dpkg -s <pkg>` – показать информацию об указанном пакете;
- `dpkg -L <pkg>` – перечислить файлы, принадлежащие указанному пакету;
- `dpkg -S <file>` – выяснить, к какому пакету относится указанный файл;

Пакетный менеджер APT

APT (advanced packaging tool) — программа для установки, обновления и удаления программных пакетов в операционных системах **Debian** и основанных на них (**Ubuntu**, **Linux Mint** и т. п.).

На сегодняшний день команда **apt** реализует почти весь функционал устаревших (но всё ещё доступных) **apt-get** и **apt-cache**.

Синтаксис построения команды

Пример:

`apt install -y vim`

Где:

- `apt` – пакетный менеджер;
- `install` – выполняемое действие;
- `-y` – ключ команды;
- `vim` – цель этой команды.

Обновление пакетов

Обновление списка пакетов:

apt update — необходимо для получения последней информации о хранящихся в репозитории пакетах.

Обновление пакетов:

apt upgrade — обновляет все установленные в системе пакеты до актуальных версий.

Информация о пакетах

- `apt search <pattern>` – полнотекстовый поиск в репозитории по паттерну;
- `apt show <pkg>` – показывает информацию о пакете (пакетах);
- `apt list` – выводит список пакетов в зависимости от ключа:
 - `--installed` – установленных;
 - `--upgradeable` – доступных к обновлению;
 - `--all-versions` – всех доступных.

Установка и удаление пакета из репозитория

- `apt install <pkg>` – устанавливает в систему пакет программного обеспечения, разрешая, по возможности, зависимости;
- `apt remove <pkg>` – удаляет файлы приложения из системы, но оставляет пользовательские файлы с настройками;
- `apt purge <pkg>` – удаляет *все* файлы приложения, включая файлы настроек;
- `apt reinstall <pkg>` – переустанавливает пакет в системе.

Полезные команды

- `apt autoremove` – удаляет неиспользуемые пакеты, например, старые версии ядра;
- `apt -f install` – попыбует починить сломанную установку пакета, например, неудовлетворенные зависимости.



Компиляция программ и сборка пакетов

Компиляция пакета из исходников

В современном мире не всегда приложение упаковывается в **готовый пакет**.

Самые последние версии ПО, к примеру, зачастую распространяются в форме **исходного кода**.

Компиляция пакета из исходников

Использование ПО из таких источников имеет некоторые **преимущества и недостатки:**

1. Вам надо самостоятельно скомпилировать приложение.
➡ В случае коллизий – самостоятельно решить появившиеся проблемы.
2. Скомпилированное из исходников ПО не считается установленным для менеджера пакетов, он его просто не видит.
➡ Удалять надо будет руками.
3. Это зачастую единственный способ получить последние или не очень популярные, но необходимые для работы программы

Команда make

Для сборки нам нужны **компиляторы** и другие инструменты. Зачастую бывают нужны утилиты **autoconf** и **automake**.

В дистрибутивах на базе Red Hat:

```
dnf install gcc make autoconf automake
```

В дистрибутивах на базе Debian:

```
apt install build-essential autoconf automake
```

Обычно в архиве с программой есть файл **configure**, выполнение которого подготовит исходные коды к **сборке**.

Иногда его нет, но есть скрипт **bootstrap** или **autogen.sh**, выполнение которого его создаст.

Команда make

`make` – дальнейшая подготовка программы к компиляции и собственно компиляция. (Сам `make` компиляцию не выполняет, он интерпретирует скрипты и вызывает компиляторы.)

`make install` – традиционно копирует файлы в конечную директорию.

`make uninstall` – встречается **крайне редко**; удалять придётся вручную.

Если вы готовы **вручную поддерживать** установленную таким образом программу, и можете гарантировать, что не будет **коллизий файлов**, можно идти этим путём.

Но в больших инфраструктурах это создаёт очень много ручной работы, поэтому **правильнее собирать пакет**.

Сборка пакета

Для создания пакета необходимо иметь не только исходные коды программы или файлы, которые надо упаковать в пакет, но и специальный файл (*spec* для rpm, *control* для deb), который описывает пакет, в том числе **последовательность действий для его сборки, конфликты, зависимости** и прочую полезную информацию, вроде **описания и имени**.

Собранный пакет дальше устанавливается при помощи менеджера пакетов.

Пример сборки пакета (на примере rpm)

1. Установить инструменты для сборки исходных кодов и пакетов:
`dnf install rpm-build rpmdevtools`
2. Скачать архив с исходным кодом и разместить его в соответствующей директории.
3. Описать пакет и процесс его сборки в соответствующем файле, например **mypkg.spec**.
4. Выполнить команду сборки пакета, например **rpmbuild**.
5. Разместить получившийся пакет в репозитории или установить его локально.



Прочие пакетные менеджеры

Пакетный менеджер npm

npm (Node Package Manager) – пакетный менеджер для [Node.js](#), программной платформы языка JavaScript.

Используется в системах, работающих на [Node.js](#) и позволяет быстро оперировать пакетами этого фреймворка.

[npm](#) устанавливается вместе с [node](#).



Поиск и получение информации о пакете

- `npm search <pattern>` – поиск пакетов в базе данных репозитория.

Ищет как по названию, так и по описанию пакета.

- `npm view <pattern>` – просмотр информации о пакете.

Установка и удаление пакетов

- `npm install <package_name>` – установка пакета локально;

Обратите внимание, в таком виде пакеты будут устанавливаться в `current work directory`, то есть в текущую рабочую директорию.

➡ Если вам надо установить пакет глобально, воспользуйтесь ключом `-g`.

- `npm uninstall <package_name>` – удаление локально установленного пакета.

Для удаления глобально установленного, воспользуйтесь ключом `-g`.

Пакетный менеджер pip

Pip – пакетный менеджер языка Python, на нем же и написанный.

Версии Python, начиная с 2.7.9 и 3.4, содержат пакет pip по умолчанию*.

Если же pip отсутствует, то его можно установить, скачав с официального сайта:

```
curl https://bootstrap.pypa.io/get-pip.py | python**
```

* Или pip3 для Python 3, если одновременно установлен 2 и 3 Python.

** Скрипт установки написан на Python, и для его исполнения нужен установленный в системе Python.



Поиск и получение информации о пакете

- `pip search <pattern>` – поиск пакетов в базе данных репозитория.

Ищет как по названию, так и по описанию пакета.

- `pip list` – список установленных пакетов;
- `pip show <package_name>` – показывает информацию о пакете.

Установка и удаление пакетов

- `pip install <package_name>` – установка выбранного пакета;
- `pip uninstall <package_name>` – удаление установленного пакета;

Пакеты устанавливаются глобально, для всей системы.

- `pip install -U` – обновление пакетов.

Пакетный менеджер Gem

Gem – пакетный менеджер языка Ruby, созданный для упрощения процесса создания, распространения и установки библиотек.

С версии Ruby 1.9 RubyGems входит в стандартный пакет.

При необходимости установить его вручную:

- скачайте пакет по ссылке: [wget](https://rubygems.org/rubygems/rubygems-3.2.8.tgz)
<https://rubygems.org/rubygems/rubygems-3.2.8.tgz>;
- распакуйте при помощи [tar](#);
- запустите установочный скрипт: [ruby setup.rb](#).

* Скрипт написан на Ruby и для его выполнения в системе должен быть установлен Ruby.



Поиск и получение информации о пакете

- `gem search -r <pattern>` – поиск пакетов в базе данных репозитория.

Ключ `-r` определяет, что поиск будет произведен в репозитории. Для поиска локально используйте ключ `-l`.

- `gem list` – список установленных пакетов.

Установка и удаление пакетов

- `gem install <package_name>` – установка выбранного пакета;
- `gem uninstall <package_name>` – удаление установленного пакета;

Пакеты устанавливаются глобально, для всей системы.

- `gem update` – обновление пакетов.



Итоги

Итоги

Сегодня мы узнали:

- что такое пакет;
- что такое официальные и сторонние репозитории;
- как работать со сторонними репозиториями;
- как пользоваться менеджерами пакетов;
- научились работать с исходными файлами;
- какие сторонние менеджеры пакетов еще есть для облегчения работы;

Домашнее задание

Давайте посмотрим ваше [домашнее задание](#).

- Вопросы по домашней работе задавайте **в чате telegram**.
- Задачи можно сдавать **по частям**.
- Зачёт по домашней работе проставляется после того, как **приняты все задачи**.

**Задавайте вопросы и
пишите отзыв о лекции!**

Андрей Тряпичников